Eberhard Karls Universität Tübingen
Mathematisch-Naturwissenschaftliche Fakultät
Wilhelm-Schickard-Institut für Informatik

Bachelor Thesis Informatics

Minimal Point Set Crossing Embedding

Bela Thrän

18.12.2024

Reviewer

Lena Schlipf
Department of Computer Science
University of Tübingen

Supervisor

Lena Schlipf
Department of Computer Science
University of Tübingen

Thrän, Bela:
Minimal Point Set Crossing Embedding
Bachelor Thesis Informatics
Eberhard Karls Universität Tübingen
Period: 16.09.2024 - 18.12.2024

Abstract

Graph drawing plays a critical role in various applications, from network visualization to VLSI (Very Large Scale Integration) design. A key challenge in this domain is the *Minimal Point Set Crossing Embedding Problem*, which seeks to map a graph $\mathcal{G}$ to a plane with a fixed set of points $\mathcal{P}$, while minimizing the number of straight-line edge crossings. Currently, common approaches include hill-climbing and simulated-annealing methods which iteratively move vertices to new positions, where the total number of crossings is reduced. We test a new approach to reduce the computational complexity. In order to do that, we also focus on the spatial arrangement of the vertices and edges rather than just the graph's abstract combinatorial structure. The main challenge for that lies in mapping an optimized graph onto our point set. Through clustering, spring embedding, and weight matching, the vertices of the graph are mapped onto the point set while the number of crossings is minimized. After the mapping is complete, the hill-climbing method is used to further optimize the graph. The hill climbing method is applied first to each cluster and then to the entire graph. The results contribute to a deeper understanding of the geometry behind graph embedding challenges.

Acknowledgements

I am very grateful to my supervisors Maximilian Pfister and Lena Schlipf. Their exceptional expertise and guidance throughout this thesis have been invaluable. I could not have wished for better supervision.
Special thanks also go to Ricarda Lepsius for her critical proofreading.

Contents

List of Figures

Chapter 1

Introduction

Graph drawing is a scientific field in computer science that tries to find a geometrical representation of an abstract object called a graph. One example is a network map. In graph drawing and computational geometry, one of the central optimization problems is the *crossing minimization problem (CMP)* [28]. Let $cr(\mathcal{G})$ denote the *minimal crossing number* of the graph $\mathcal{G}$, which represents the smallest number of edge crossings achievable over all drawings in the plane of $\mathcal{G}$. The crossing minimization problem seeks to determine a drawing with minimal number of edge crossings. The problem of minimizing edge crossings is of great interest due to its wide applications in areas such as VLSI circuit design [6], network visualization [3], and data organization [11]. The application of VLSI circuits has a strict metric of how good a solution is, namely the number of crosses. In contrast, the metric for evaluating the suitability of a graph drawing for human perception is often ambiguous. There exist many metrics [3], however one of the main metrics is also the minimization of the number of crosses.

Background on Crossings in Graph Theory

1. **Zarankiewicz Conjecture**
   The Zarankiewicz Conjecture [7] focuses on determining the crossing number of bipartite graphs. Specifically, the conjecture seeks to find the minimum number of crossings required in a drawing of a complete bipartite graph $K_{m,n}$, where the vertices are partitioned into two sets of m and n vertices, with edges only between sets. The conjecture postulates that the minimum number of edge crossings in any drawing of $K_{m,n}$ in the plane is bounded by:

$$\mathrm{cr}(K_{m,n}) \geq \left\lfloor \frac{m}{2} \right\rfloor \left\lfloor \frac{m-1}{2} \right\rfloor \left\lfloor \frac{n}{2} \right\rfloor \left\lfloor \frac{n-1}{2} \right\rfloor$$

This formula provides an asymptotic lower bound for the number of crossings in any drawing of $K_{m,n}$ (complete bipartite graph).

2. **Crossing Lemma**
   The Crossing Lemma [23] provides a lower bound on the number of edge crossings in any drawing of a graph with a large number of edges relative to the number of vertices. It states that for a graph $\mathcal{G}$ with n vertices and m edges, if $m > 4.5n$, then the crossing number $\text{cr}(\mathcal{G})$ is at least:

   $$\text{cr}(\mathcal{G}) \geq \frac{e^3}{64n^2}$$

   It is a powerful tool in analyzing dense graphs, as it imposes a condition on the minimum crossings required based on the number of vertices and edges, suggesting that high-density graphs cannot avoid crossings altogether.

   For $m > 6.77n$, [5] proofed an improved constant with

   $$\text{cr}(\mathcal{G}) \geq \frac{e^3}{27.48 \cdot n^2}$$

   regarding the Crossing Lemma.

3. **Planar Separator Theorem**
   For any n-vertex planar graph $\mathcal{G}$, this theorem states that the removal of $\mathcal{O}(\sqrt{n})$ vertices of $\mathcal{G}$ can partition the graph into two disjoint subgraphs, where each subgraph has at most $\frac{2n}{3}$ vertices [24]. This helps in efficiently solving problems on planar graphs by reducing the complexity of the remaining graph components.

4. **Fáry's Theorem**
   Fáry's theorem states that any simple, planar graph can be drawn without crossings so that its edges are straight line segments [20]. That is, the ability to draw graph edges as curves instead of as straight line segments does not allow a larger class of graphs to be drawn [4].

Related Works

Research on minimal point set crossing embedding spans several decades and draws from various subfields of graph theory, computational geometry, and visualization. In this section, we review key contributions and advancements relevant to crossing minimization, point set embeddings, straight line drawings, and practical applications.
The study of *point set embeddings* emerged as a significant focus in the 1980s

and 1990s. Researchers began examining how to embed plane graphs [10]. In parallel, the concept of the *crossing number* was introduced, formalizing the idea of minimizing the number of edge intersections.
In recent years, there have been notable advancements in algorithms for crossing minimization. Given the computational complexity of the crossing number problem, researchers have developed heuristics that offer practical solutions for larger graphs. One example would be star insertion [9] which is concerned with topological crossing minimization. Besides that force-directed drawing algorithms [13, 21], which we will also make use of, have been developed. For heuristic algorithms exact solutions remain challenging. Another approach to solving crossing minimization problems are genetic algorithms [2, 15] and machine learning algorithms [19]. In practice optimization algorithms like simulated annealing [22, 29] and hill-climbing [27] are often used. They iteratively move the vertices of $\mathcal{G}$ to different regions of $\mathcal{G}$ while minimizing the edge intersections.

Related Problems

There are a lot of NP-hard subproblems which are closely related to our problem.

I **Graph Crossing Number**
The problem of determining the minimum number of edge crossings required to draw a graph in the plane. Finding the crossing number for arbitrary graphs is a classic NP-hard problem [18].

II **Straight-Line Crossing Minimization**
In straight-line drawings, finding the minimum crossing configuration while keeping edges straight is NP-hard [28]. This variant of the graph crossing number problem (see I) is important for applications needing visually intuitive layouts.

III **Maximum Planar Subgraph Problem**
The task of finding the largest planar subgraph of a given non-planar graph is NP-hard [8]. This is significant in applications where an approximate planar representation is sought.

IV **Weighted Crossing Number Problem**
When edges have weights, the problem involves minimizing the total weight of crossings, making it even more challenging than the unweighted crossing number problem (see I). Consequently this problem is also NP-hard [28].

V **Clustered Graph Drawing**
Drawing clustered graphs (where vertices are grouped into clusters) with

minimal crossings is NP-hard [16], particularly when vertices within clusters must respect fixed positions.

Problem Statement

Previously, the crossing number, denoted $\mathrm{cr}(\cdot)$, was defined. We now introduce the notion of the *crossing value*, denoted $cv(\cdot)$, which represents the number of edge crossings occurring in a given graph drawing.
In particular, a point set drawing involves placing a graph's vertices ($\mathbf{V}$) on predetermined points $\mathcal{P}$ in the plane. The edges can then be represented as straight-line segments. Despite advances in algorithms for crossing minimization [12, 9], finding efficient methods that yield optimal or near-optimal solutions for arbitrary graphs remains an open challenge, particularly in the context of fixed point sets. The optimization problem of finding the injective mapping $\phi(\cdot)$ from the vertices ($\mathbf{V}$) to the points $\mathcal{P}$ which leads to the minimal crossings number $\mathrm{cr}(\mathcal{G})$ in $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ is called *minimal point set crossings embedding problem*. The Graph Drawing Symposium tried to tackle this problem with their Graph Drawing Challenge 2024: Given a graph $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ and a point set $\mathcal{P}$ with $|\mathcal{P}| \geq |\mathbf{V}|$, find an injective mapping:

$$\phi : \mathbf{V} \to \mathcal{P}$$

which minimizes the number of edge-crossings in the resulting straight-line graph drawing.

$$\min_{\phi} cv(\mathcal{G}, \phi)$$

Counting Crossings

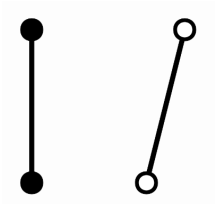
To gain a deeper understanding of the problem we are going to take a deeper look on how the crossings are counted in the Graph Drawing Challenge. They introduce three different crossing values that two arbitrary edges can have.

1. Two arbitrary edges e_1 and e_2 do not intersect at all (see Figures 1.1a and 1.1b). This is not counted as a crossing. Consequently e_1 and e_2 have a crossing value of zero.

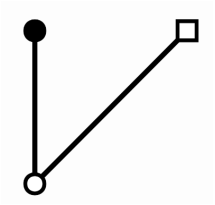
2. Two arbitrary edges e_1 and e_2 intersect in a non-collinear manner (see Figure 1.1c). This is counted as a normal crossing where e_1 and e_2 have a crossing value of one.

3. Two arbitrary edges e_1 and e_2 intersect in a collinear manner (see Figure 1.2). These types of intersection are penalized with a crossing value of

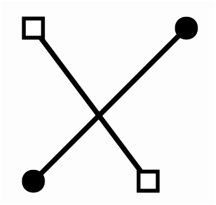
$|\mathbf{V}|$. Therefore, it is essential to especially avoid collinear crossings. The edges e_1 and e_2 are counted as collinear if at least one node of one edge is located on the other edge (see Figures 1.2a to 1.2d).



(a) Case 1a: no common point, so these two edges have a cross value of zero.

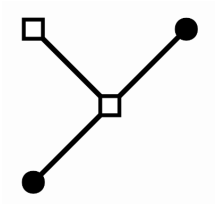


(b) Case 1b: the only common point is the shared endpoint of both edges (white circle), so these two edges have a cross value of zero.

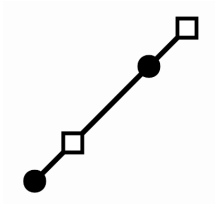


(c) Case 2: the edges cross in a not collinear fashion, so these two edges have a cross value of one.

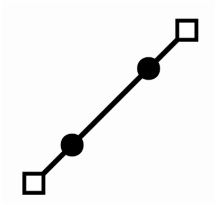
Figure 1.1: Illustrations of different cases with crossing value zero or one.



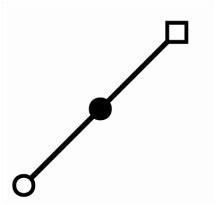
(a) Case 3a: the lower white square lies on top of the edge connecting the black dots, so these two edges have a cross value of n.



(b) Case 3b: the lower white square lies on top of the edge connecting the black dots (and the upper black dot lies on top of the edge connecting the white squares), so these two edges have a cross value of n.



(c) Case 3c: both black dots lie on top of the edge connecting the white squares, so these two edges have a cross value of n.



(d) Case 3d: though the two edges share a vertex (white circle) the black dot lies on top of the edge connecting the shared vertex to the white square, so these two edges have a cross value of n.

Figure 1.2: Illustration of different cases with crossing value n.

The paper is structured as follows: In Chapter 2, we describe the used methodologies employed for achieving minimal point set crossing embeddings. The primary data obtained and results of our analyses are then presented in Chapter 3.

Finally, Chapter 4 offers a discussion of the findings and their implications, concluding with a brief outlook on future research directions.

Chapter 2

Methods

The core methodology of our approach is outlined in Algorithm 1. The primary challenge lies in mapping an optimized graph $\mathcal{G}^O$ to a corresponding point set $\mathcal{P}$. To address this in a computationally efficient manner, the graph $\mathcal{G}$ is partitioned into a set of clusters $\mathcal{C} = \{\mathcal{C}_i\}$, where each $\mathcal{C}_i$ represents an individual graph cluster. Similarly, the point set $\mathcal{P}$ is partitioned into point clusters $\mathcal{P}_C = \{\mathcal{P}_{Ci}\}$, aligning with the graph clusters $\mathcal{C}$. This enables an initial approximation of the positions for the point clusters.

Subsequently, the graph cluster set $\mathcal{C}$ is optimized to $\mathcal{C}^O$, reducing edge crossings, and the optimized clusters are mapped to their estimated positions in $\mathcal{P}_C$. Each graph cluster $\mathcal{C}_i$ is then individually optimized to $\mathcal{C}_i^O$ and mapped to its respective point cluster $\mathcal{P}_{Ci}$.

To further reduce edge crossings, an iterative hill-climbing optimization is employed, initially at the level of all individual graph clusters $\mathcal{C}_i$ and subsequently at the global level for the entire graph $\mathcal{G}$.

2.1 Graph Clustering

The graph $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ is clustered to reduce the complexity of the problem. After the clustering of $\mathcal{G}$, every graph cluster $\mathcal{C}_i$ still needs to be mapped to its corresponding point set $\mathcal{P}_{Ci}$. For the graph clustering we use the clusterer module from OGDF based on [1]. The clusterer module presents two different clustering approaches:

1. Recursive Clustering
   In recursive clustering, the clustering process is repeated on the graph or its subcomponents. This process iteratively removes weak edges. The strength metric of an edge can be interpreted as a measure of its contribution to the cohesion of its neighborhood. The edge strength is recom-

Algorithm 1 Basic Fuctionality

Require: $\mathcal{G} = (V, E)$ and $\mathcal{P}$
Ensure: $|\mathcal{P}| \geq |V|$
1: $\mathcal{C} \leftarrow$ clustering($\mathcal{G}$)
▷ Cluster $\mathcal{G}$ into k clusters $\mathcal{C} \leftarrow \bigcup_{i=1}^{k} \mathcal{C}_i$
2: $\mathcal{G}_W \leftarrow$ createWeightedClusterGraph($\mathcal{C}$)
▷ Each vertex of $\mathcal{G}_W$ represents one cluster
3: $\mathcal{P}_C \leftarrow$ localClustering($\mathcal{P}$, sameSize)
4: **for** $\mathcal{P}_{Ci}$ **in** $\mathcal{P}_C$ **do**
5: $Centroid_i \leftarrow$ center($\mathcal{P}_{Ci}$) ▷ Compute center of every point cluster
▷ Used to approximate location of point clusters
6: **end for**
7: $\mathcal{G}_W^O \leftarrow$ springEmbedding($\mathcal{G}_W$)
8: $\mathcal{G}_W \leftarrow$ minimumDistanceMatching($\mathcal{G}_W^O$, *centroids*)
9: $\mathcal{P}_C \leftarrow$ localClustering($\mathcal{P}$, sizes($\mathcal{G}_W$))
▷ Iterate through every graph cluster
10: **for** i **in range** $\mathcal{C}$.size() **do**
11: $\mathcal{C}_i^O \leftarrow$ springEmbedding($\mathcal{C}_i$)
12: $\mathcal{C}_i \leftarrow$ minimumDistanceMatching($\mathcal{C}_i^O$, $\mathcal{P}_{Ci}$)
13: $\mathcal{C}_i \leftarrow$ localBaseline($\mathcal{C}_i$, $\mathcal{G}_W$, $\mathcal{P}_{Ci}$)
14: **end for**
15: $\mathcal{G} \leftarrow$ map(C)
16: $\mathcal{G} \leftarrow$ globalBaseline($\mathcal{G}$)

puted after every recursion. The clusters are refined recursively until the clustering index reaches the stop index.

2. Non-Recursive (Iterative) Clustering
   The non-recursive graph clustering uses multiple thresholds in one pass. The connectivity thresholds are based on precomputed strengths and can be set automatically. Since the clusters are created in one step and no refinement is needed, the non-recursive graph clustering is significantly faster.

Recursive clustering is typically more thorough but slower, while non-recursive clustering is faster but may not be as precise.

2.1.1 Weighted Cluster Graph ($\mathcal{G}_W$)

Heavily connected clusters should be placed close to each other to reduce the overall number of crossings. To achieve that, we create a weighted cluster graph $\mathcal{G}_W$ from $\mathcal{C}$. This allows us to optimize the positions of the clusters later on.

$$\mathcal{G}_W = (V_{\mathcal{GW}}, E_{\mathcal{GW}}, w)$$

$$V_{\mathcal{GW}} = \bigcup_{i=1}^{k} i \quad \text{with} \quad |V_{\mathcal{GW}}| = k \leftarrow \text{Number of clusters}$$

$$E_{\mathcal{GW}} = \{(i,j), w(i,j)\}$$

$$w(i,j) = |\{\forall (u,v) \in \mathbf{E} \;\mid\; (u \in \mathcal{C}_i \wedge v \in \mathcal{C}_j) \;\vee\; (v \in \mathcal{C}_i \wedge u \in \mathcal{C}_j)\}|$$

Each node in $\mathcal{G}_W$ represents one graph cluster $\mathcal{C}_i$. The weight of an edge between two nodes i and j in $\mathcal{G}_W$ is derived from the total number of edges between $\mathcal{C}_i$ and $\mathcal{C}_j$.

2.2 Local Point Clustering

To point cluster the point set $\mathcal{P}$, we have to pay attention to some constraints. The number of point clusters and the size of each point cluster are given from $\mathcal{C}$. In order to match these constraints we need a point clustering algorithm for the point set with k point clusters where each point cluster $\mathcal{P}_{Ci}$ needs to have at least as many points as its matching graph cluster $\mathcal{C}_i$ has vertices. For the point clustering reducing the Euclidean Distance between clusters is not always beneficial. Instead the key focus of the local point cluster algorithm is as follows.

$$conv(\cdot) \leftarrow \text{convex hull of a set of points}$$

$$\forall\; p \in \text{conv}(\mathcal{P}_{Ci}) \mid p \in \mathcal{P}_{Ci}$$

All points within a convex hull of an arbitrary point cluster $\mathcal{P}_{Ci}$ should be assigned to that point cluster $\mathcal{P}_{Ci}$. If a point $p \notin \mathcal{P}_{Ci}$ lies within the convex hull of $\mathcal{P}_{Ci}$, it can lead to edge crossings when constructing the drawing of $\mathcal{C}_i$. This is because the convex hull forms the outer boundary of $\mathcal{P}_{Ci}$. To illustrate this, consider the following example:
Suppose $\mathcal{C}_i$ is a complete graph with $|\mathcal{C}_i| = |\mathcal{P}_{Ci}|$ and $p \in \mathcal{P}_{C_j}$ is the only point $\notin \mathcal{P}_{Ci}$ that lies within the convex hull of $\mathcal{P}_{Ci}$. Consequently the node $v \in \mathcal{C}_j$ mapped on p will have at least one crossing for every edge $e = (u, v)|u \in \mathcal{C}_j$.

In order to match these constraints we developed a point cluster algorithm. Our proposed local point cluster algorithm clusters the points from eight different directions. The points are sorted in regard to their Manhattan distance from each corner (NE, SE, SW, NW) and in regard to their Manhattan distance to the sides (N, E, S, W). This leads to eight sorted direction vectors. Now the first k points from a direction vector, which are not already in a cluster, are added to a new cluster. Afterwards we iterate to the next direction vector.

Algorithm 2 Local Clustering

Require: int numberOfClusters **and** array sizesOfClusters
1: **for** i **in range** numberOfClusters **do**
2: direction $\leftarrow i\%8$
3: currentDirection $\leftarrow$ directions[direction]
▷ currentDirection is of size $\mathcal{P}$
4: **for** Point p **in** $\mathcal{P}$ **do**
5: **if** p is not used **then**
6: $\mathcal{P}_{Ci}$.add(p) ▷ add p to current point cluster
7: **end if**
8: **if** $\mathcal{P}_{Ci}$.size() **equals** clusterSizes$_i$ **then**
9: **break**
10: **end if**
11: **end for**
12: **end for**

The local cluster algorithm is used twice:

1. After the computation of the weighted cluster graph ($\mathcal{G}_W$), the vertices of the $\mathcal{G}_W$ need to be mapped onto a point set. Heavily connected graph clusters should be close to each other, whereas not connected graph clusters do not need to be close by. To approximate the point cluster areas we use the local cluster algorithm and compute the centroids of each point cluster. In Figure 2.1 the local cluster algorithm is illustrated with 21 points and seven point clusters. Therefore the centroids are computed

from seven point clusters with three points each. The centroids are then used as a point set for the mapping of $\mathcal{G}_W$. Consequently we retrieve an order in which the second local clustering should take place.

2. The final local point clustering is used to cluster all the points in regards to the actual cluster sizes. If the number of given vertices equals the number of given points, the cluster sizes of the graph clustering can be used for the local point clustering. Otherwise, if the number of given vertices is smaller than the number of points, the cluster sizes of the graph cluster are equally increased until their sum matches the given number of points. The order in which the local clustering takes place depends on the first local clustering.

2.2.1 Runtime

The runtime of the local clustering algorithm is $O(n \log n + nk)$ where k is the number of clusters and n the number of points.

2.3 Spring Embedding

Before each clustered graph is mapped onto a point set it is optimized with the spring embedding layout algorithm by Kamada and Kawai [21]. We use the implementation from OGDF for the spring embedding. This algorithm interprets each edge as a "spring" with a desirable length. They regard the optimal layout of vertices as the state in which the total spring energy of the system is minimal. This has been shown to reduce the number of edge crossings in a graph.

2.3.1 Spring Embedding in Weighted Cluster Graph $\mathcal{G}_W$

To apply the Kamada and Kawai spring embedding [21] to weighted graphs, certain modifications are required. In $\mathcal{G}_W$, higher edge weights should result in closer proximity between the corresponding vertices. This ensures that graph clusters with numerous strong connections are positioned close together, reducing the likelihood of edge crossings.

2.3.2 Spring Embedding for each Cluster

The most effective way to reduce the number of crossings with the spring embedding in an unweighted graph is by setting an equal desired edge length for every edge.

2.4 Minimum Distance Matching

The motivation behind the minimum distance matching is to map an optimized graph $\mathcal{G}^O = (V_\mathcal{G}, E_\mathcal{G})$ with reduced crossings to a given point set $\mathcal{P}$. Generally, we can assume that $\mathcal{G}^O$ is either a graph cluster $\mathcal{C}_i$ (see 2) or a weighted cluster graph $\mathcal{G}_W$ (see 1). To create a meaningful mapping, the bounding boxes of $V_\mathcal{G}$ and $\mathcal{P}$ are aligned as shown in Figure 2.2b. Since $\mathcal{G}^O$ is already optimized, $E_\mathcal{G}$ can be neglected for the minimum distance matching.

$$m = V_\mathcal{G} \quad \text{and} \quad n = \mathcal{P}$$

$$\mathcal{K}_{m,n} = (V_\mathcal{K}, E_\mathcal{K})$$

$$V_\mathcal{K} = V_\mathcal{G} \cup \mathcal{P} \quad \text{and} \quad E_\mathcal{K} = \{(u, v), w(u, v) \mid u \in V_\mathcal{G}, v \in \mathcal{P}\}$$

The complete bipartite graph $\mathcal{K}_{m,n}$ is used to map $V_\mathcal{G}$ to $\mathcal{P}$ (see 2.2c). For an edge $e = (u, v)$, the weight $w(u, v)$ represents the distance between the node n and the point u. Later on, the maximum weighted matching function (see 2.4.1) from the boost library is used. Consequently the following weight function is used:

$$w(u, v) = num \cdot \frac{1}{\sqrt{(v.x - u.x)^2 + (v.y - u.y)^2} + 1} \text{ with } num \in \mathbb{N}$$

The weight of each edge (u, v) is now the inverse of the Euclidean distance, multiplied by num, a relatively high natural number to gain a better distribution of the weight values. This results in short edges with relatively high weights and long edges with relatively low weights. Now, the maximum weighted matching will lead to a minimum distance matching as demonstrated in Figure 2.2d.

2.4.1 Maximum Weighted Matching

Our algorithm uses the boost implementation of Edmond's maximum weight matching algorithm [14]. The used implementation is rather unstable and does not work for all weight initializations. Therefore, we use a greedy minimum distance matching algorithm as shown in Algorithm 3 as a fallback solution. Edmond's maximum weight matching algorithm has a runtime of $O(|V_\mathcal{K}|^3) = O((|V_\mathcal{G}|+|\mathcal{P}|)^3)$. Because the number and sizes of graph cluster are very hard to estimate (see 2.1), the worst runtime when $\mathcal{C}$ equals $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ (there is only one cluster) is $O((|\mathbf{V}| + |\mathcal{P}|)^3)$. Luckily this case is very unlikely and we can assume a better runtime for this algorithm. It is worth mentioning that there is a better algorithm for the maximum weight matching problem [17] which has a runtime of $O(nm + n^2 \log(n))$. But it relies on an efficient algorithm for solving nearest ancestor problem on trees, which is not provided in the Boost

C++ libraries.

2.4.2 Greedy Minimum Distance Matching

As mentioned in Section 2.4 the boost maximum weighted matching function is limited to single precision numbers and a very specific weight initialization. We consequently need a fallback solution. For that we use the greedy minimum distance matching (see 3). This algorithm will also find a perfect matching,

Algorithm 3 Greedy minimum distance matching

```
Require: Edges E with weights w(e)
1: E ← sortAscending(E) by w(e)
2:
3: for e in E do
4:     if nodes(e) are not in matching then
5:         Use e for matching
6:     end if
7: end for
```

because it is used on the complete bipartite graph $\mathcal{K}$. Another advantage of the *greedy minimum distance matching* is its speed with $O(\mathbf{E} \cdot \log(|\mathbf{E}|)$, where $|\mathbf{E}| = |\mathbf{V}| \cdot |\mathcal{P}|$.

2.4.3 Application

As mentioned earlier there are two possible optimized input graphs $\mathcal{G}_O$ for the minimum distance matching.

1. $\mathcal{G}_W$ - weighted cluster graph
   In line 8 of the basic functionality algorithm the minimum distance matching is called with $\mathcal{G}_W$ and the centroids. Here, the position of the clusters is matched to the approximated cluster positions.

2. $\mathcal{C}_i$ - graph cluster
   In line 12 of the basic functionality algorithm the minimum distance matching is called with $\mathcal{C}_i$ and $\mathcal{P}_{Ci}$. Figure 2.2 illustrates a visualized example. Here, one graph cluster is mapped to its according point cluster.

2.5 Hill-Climbing Method

The hill-climbing approach to reduce the number of crossings in a graph $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ consists of iteratively moving nodes to random positions where they lead to less crossings. Due to our constraint of a fixed point set $\mathcal{P}$, we are able to move the nodes more determined. Iteratively a node is either swapped with another node or moved to a free point. Consequently, if $|\mathbf{V}| = |\mathcal{P}|$ we can only apply swapping operations to the nodes. Because after each swapping or moving operation the number of crossings in $\mathcal{G}$ needs to be computed, an efficient method to compute the number of crossings is substantial.

2.5.1 R-tree

An R-tree is a spatial data structure that is particularly effective for efficiently indexing multi-dimensional information, such as geographical coordinates. For our application only two dimensional R-Trees are of interest. In the context of finding crossings in a graph, an R-tree can significantly reduce the computational complexity of detecting intersections. The following attributes of an R-tree are useful for crossing detection in geometric graphs.

1. **Store Edge Bounding Boxes in the R-tree**
   Each edge in the graph can be represented by its *bounding box*, which is the smallest rectangular area containing the edge. By inserting each edge's bounding box into the R-tree, we can spatially index the edges for quick access based on their locations.

2. **Efficient Range Queries**
   When testing for crossings, we need to find pairs of edges that might intersect. The R-tree allows us to perform efficient *range queries* to retrieve only those edges whose bounding boxes overlap with the bounding box of the current edge being tested. This reduces the number of potential edge pairs we need to test directly.

3. **Avoiding Unnecessary Comparisons**
   By leveraging the hierarchical nature of the R-tree, unnecessary comparisons are avoided. Only edges that are spatially close enough to potentially intersect are considered, thus minimizing computational effort.

The R-tree minimizes the number of edge pairs needing intersection testing, leading to significant performance improvements for large graphs.

2.5.2 Local Hill-Climbing

The local hill-climbing approach is used in line 13 of the basic functionality algorithm. The main motivation behind the local hill-climbing approach is to minimize the crossings of a graph cluster $\mathcal{C}_i$, while considering the other graph clusters. The local hill-climbing method reduces computational overhead because not as many vertices and edges need to be taken in consideration as during the global hill-climbing approach (see 2.5.3). To compute the number of crossings efficiently we use the R-Tree structure (see 2.5.1). From our graph cluster $\mathcal{C}_i = (V_i, E_i)$ we are creating a weighted graph $\mathcal{C}_i^W$ to reduce the crossings of $\mathcal{C}_i$ in regards to the other clusters as well. An example of $\mathcal{C}_i^W$ is visualized in Figure 2.4b.

$$\mathcal{C}_i^W = (V_i^W, E_i^W, w)$$
$$V_i^W = \{j \neq i \;\; \forall j \in |\mathcal{C}|\} \cup V_i \quad \text{and} \quad E_i^W = \{(j,v), w(j,v)\}$$
$$\{j,v\} = \{(j,v) \;\; \forall j \in V_{\mathcal{GW}} \mid j \neq i, v \in V_i\} \cup E_i$$

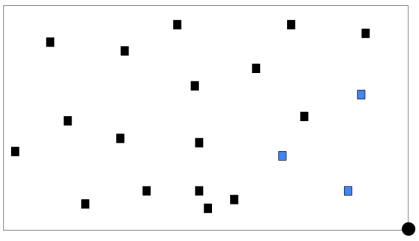
The weight of each edge in $\mathcal{C}_i^W$ is calculated as follows:

$$w(j,v) = \begin{cases} 1 & \text{if } (j,v) \in E_i \\ |\{(u,v) \;\; \forall u \in V_j \mid (u,v) \in \mathbf{E}\}| & \text{otherwise} \end{cases}$$

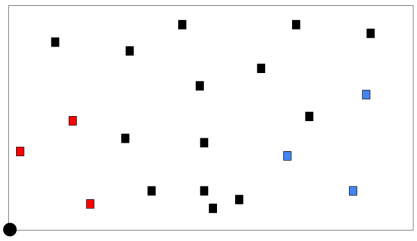
The weight of an edge in $\mathcal{C}_i^W$ resembles the number of edges between a node v in V_i and a graph cluster $\mathcal{C}_j$. This allows swapping and moving the vertices V_i to reduce the number without the need to compute all the crossing within the entire graph $\mathcal{G} = (\mathbf{V}, \mathbf{E})$. The crossing value of two edges is calculated by multiplying the weights of both edges. During the hill-climbing approach a common problem are local extremes as shown in 2.5d. A partial solution for that is to occasionally swap or move vertices even if the number of crossings remains the same. Unfortunately is this only a partial solution, a more thorough analysis will follow later in Chapter 3.

2.5.3 Global Hill-Climbing

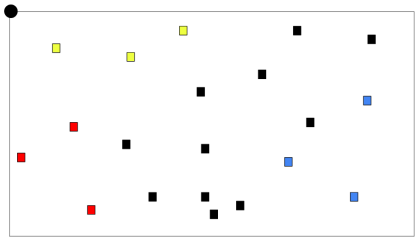
The global hill-climbing approach is used in line 16 of the basic functionality algorithm. The hill-climbing algorithm is applied to the entire graph $\mathcal{G}$, leveraging the R-tree structure to optimize the process.



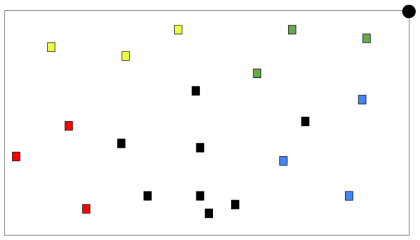
(a) 1. Iteration: starting on the south east (SE) corner and the three closest points to that corner are added to the first cluster.



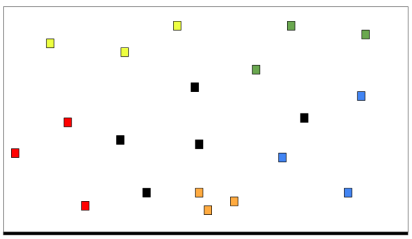
(b) 2. Iteration: continuing on the south west (SW) corner and the three closest points to that corner are added to the second cluster.



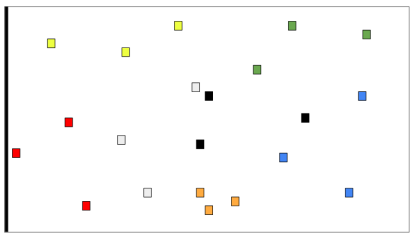
(c) 3. Iteration: on the north west (NW) corner where we again create a new cluster of size three.



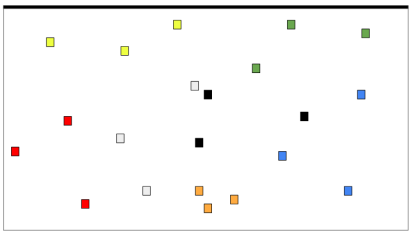
(d) 4. Iteration: the last corner (NE) creates the fourth cluster. Here we can observe that one point is already occupied. Therefore the algorithm just adds the next closest point instead.



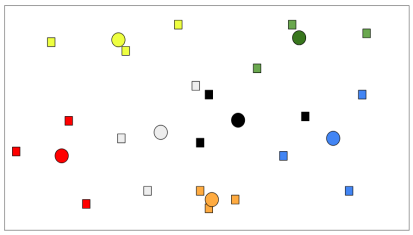
(e) 5. Iteration: clustering in regard to the south axis.



(f) 6. Iteration: clustering in regard to the east axis

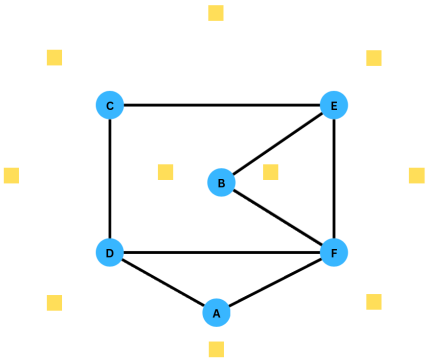


(g) 7. Iteration: clustering in regard to the north axis. The last clustering just equals all the points that are not already in a cluster.

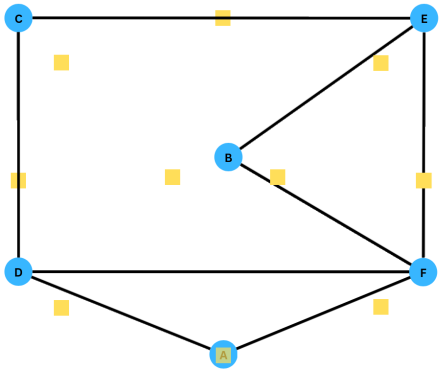


(h) After all clusters are computed the centroids of all clusters are calculated. The centroids are then used as a point set for the weighted cluster graph.

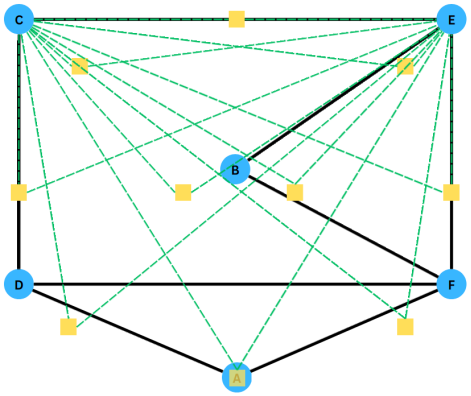
Figure 2.1: Illustration of the local clustering algorithm with same sizes to compute the point set for the weighted cluster graph ($\mathcal{G}_W$).



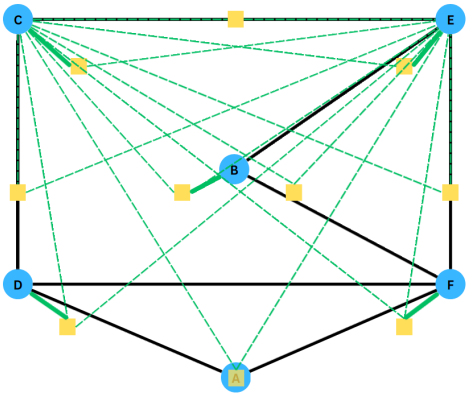
(a) Given graph $\mathcal{C}_i = (V_i, E_i)$ and $\mathcal{P}_{Ci}$ from 2.3b.



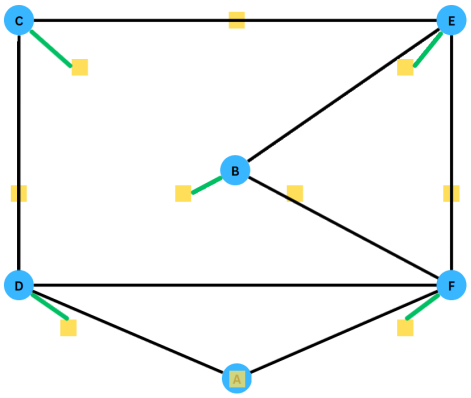
(b) In order to create a meaningful mapping from $\mathcal{C}_i$ to $\mathcal{P}_{Ci}$ we align the vertices of $\mathcal{C}_i$ in regard to the points of $\mathcal{P}_{Ci}$.



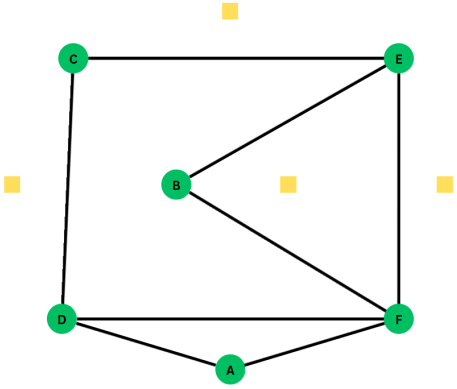
(c) Creating a complete bipartite graph $K_{m,n}$ with $m = |V_i|$ and $n = |\mathcal{P}_{Ci}|$. The edges of $K_{m,n}$ are marked green only for the vertices C and E.



(d) The thick green lines resemble the solution of the minimum distance matching.

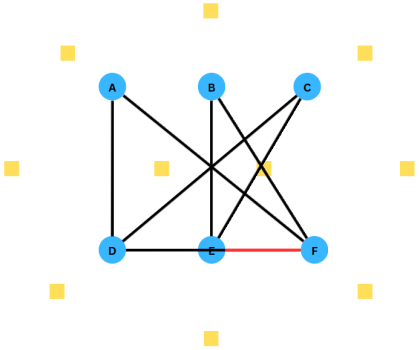


(e) Each point is mapped to exactly one point.

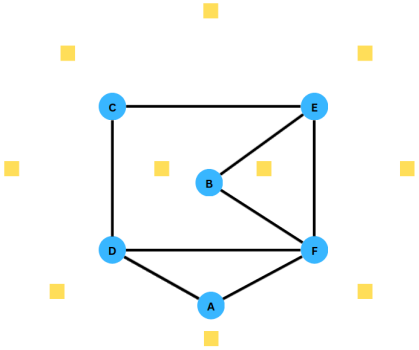


(f) Assign vertices the location of their mapped point.

Figure 2.2: Illustration of the minimum distance matching with a graph cluster $\mathcal{C}_i$ and a point cluster $\mathcal{P}_{Ci}$ as input.

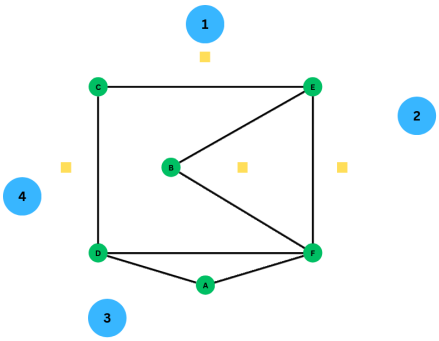


(a) The Figure illustrates a graph cluster $\mathcal{C}_i$ of $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ and a point cluster $\mathcal{P}_{Ci}$. The edge (E,F) is on top of (D,F) which results in a crossing penalty of $|\mathbf{V}|$.

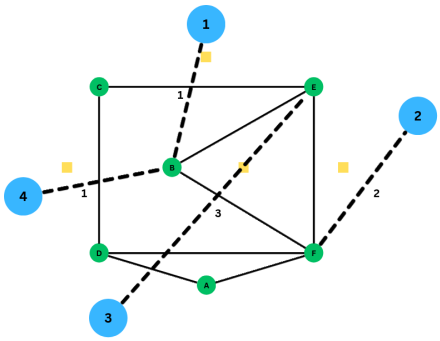


(b) Optimized $\mathcal{C}_i$ after the application of the spring embedding. The number of crossings was reduced to zero, the optimal solution. This is not always the case when applying the spring embedding.

Figure 2.3: Illustration of the Kamada and Kawai spring embedding.

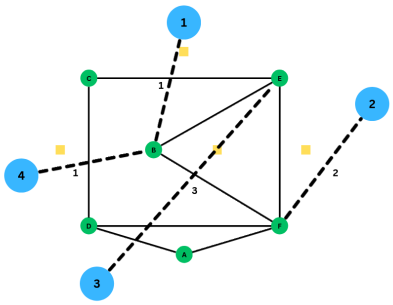


(a) The Figure illustrates the matched graph cluster $\mathcal{C}_5$ from Figure 2.2f, where the nodes are shown in green. The centroids of the graph clusters $\mathcal{C}_1$ to $\mathcal{C}_4$ are represented as blue nodes.
$|\mathcal{C}| = 5 \leftarrow$ There are 5 clusters in total.

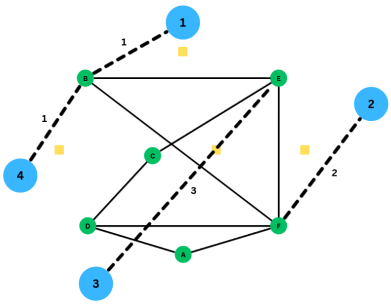


(b) To incorporate the other clusters as well, we create a weighted graph $\mathcal{C}_i^W$. In $\mathcal{C}_i^W$ all the edges within $\mathcal{C}_i$ (continuous thin lines) are weighed with one. The edges between a node v of $\mathcal{C}_i$ and a graph cluster $\mathcal{C}_j$ (thick dot lines) are weighed with the number of edges from that graph cluster $\mathcal{C}_j$ to the node v. Edges with weight zero are not drawn.

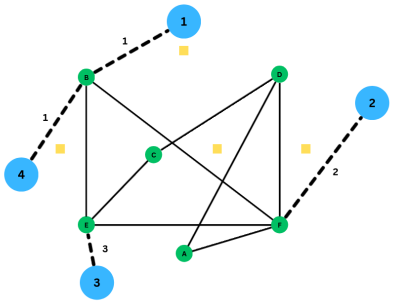
Figure 2.4: Creating $\mathcal{C}_i^W$ in preparation of the local hill-climbing approach.



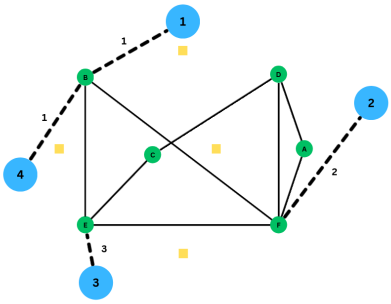
(a) $\mathcal{C}_i^W$ from Figure 2.4b has eleven crossings. The value of one crossing is calculated by multiplying the weight of both edges of the crossing.



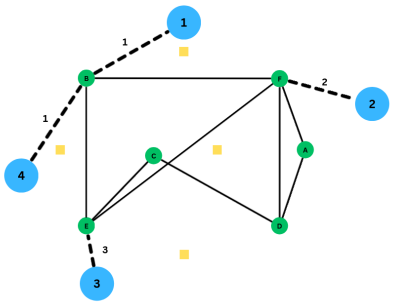
(b) The nodes C and B are swapped. This operation leads to a reduction in crossing from eleven to ten, where the edge (3, E) with weight three is responsible for nine crossings.



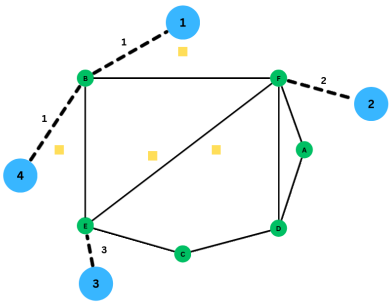
(c) By swapping the nodes E and D the crossing number is reduced to 3.



(d) After moving the node A there is only one crossing left. We now can recognize that there is no swapping or moving operation that will decrease the number of crossings further.



(e) Swapping the nodes D and F did not result in a decrease of crossings. The crossing number is still one.



(f) After moving the node C to another point, the number of crossings in $\mathcal{C}_i^W$ is reduced to zero. We therefore have reached an optimal solution.

Figure 2.5: Illustration of the local hill-climbing approach.

Chapter 3

Results

The primary objective of this research is to develop and evaluate a minimal point set crossing drawing algorithm that optimally minimizes edge crossings in a geometric graph drawing. The study aims to validate the algorithm's efficiency and effectiveness through experimental comparisons, demonstrating its ability to handle complex graph structures while preserving computational feasibility.

3.1 Experiments

Experimental Setup

The experiments for this thesis were conducted on a system with an `11th Gen Intel Core i5-1135G7 CPU` running at a base frequency of `2.40 GHz`, featuring `4 cores and 8 threads`. The processor has an `8 MB cache` and supports advanced instruction sets, including `AVX2` and `AVX-512`, optimizing performance for computation-heavy tasks.

Algorithm Stages

For a better understanding of the experiments a brief recap of stages from our proposed algorithm:

Notation	Description
$\phi(\mathcal{C}^O)$	The graph after spring optimization and minimum distance matching of every cluster. This also includes the optimization and mapping of the weighted cluster graph.
$\phi_{local-HC}(\phi(\mathcal{C}^O))$	The application of the local hill-climbing algorithm to $\phi(\mathcal{C}^O)$.
$\phi_{global-HC}(\phi_{local-HC}(\phi(\mathcal{C}^O)))$	The application of the global hill-climbing technique after the local hill-climbing. The final step of our algorithm.

Table 3.1: Steps of the graph optimization and mapping process.

To gain a deeper understanding the following stages are also valuable for the evaluation of our proposed algorithm.

Notation	Description
$\phi_{random}(\mathcal{G})$	A random mapping from vertices to points. Illustrates a starting point before any optimization technique.
$\mathcal{G}^O$	The entire graph after optimization (without clustering) with the spring embedding.
$\phi(\mathcal{G}^O)$	The graph drawing after the vertices of the entire optimized graph (without clustering) are mapped onto the point set with the minimum distance matching.
$\phi_{global-HC}(\mathcal{G})$	The global hill-climbing technique without any preprocessing steps, therefore starting with $\phi_{random}(\mathcal{G})$ as input.

Table 3.2: Descriptions of various graph drawing notations.

3.1.1 Time Complexity

First we will examine the time complexity of the proposed algorithm. The performance in crossing minimization problems is mostly measured with respect to the time an algorithm needs to effectively minimize the edge intersections. This underlines the importance of this Experiment illustrated in Figure 3.1. We compare the global hill-climbing (orange line) on its own to our proposed algorithm, including the performance of the clustering and mapping process (brown point), the local hill-climbing (blue) and the final global hill-climbing

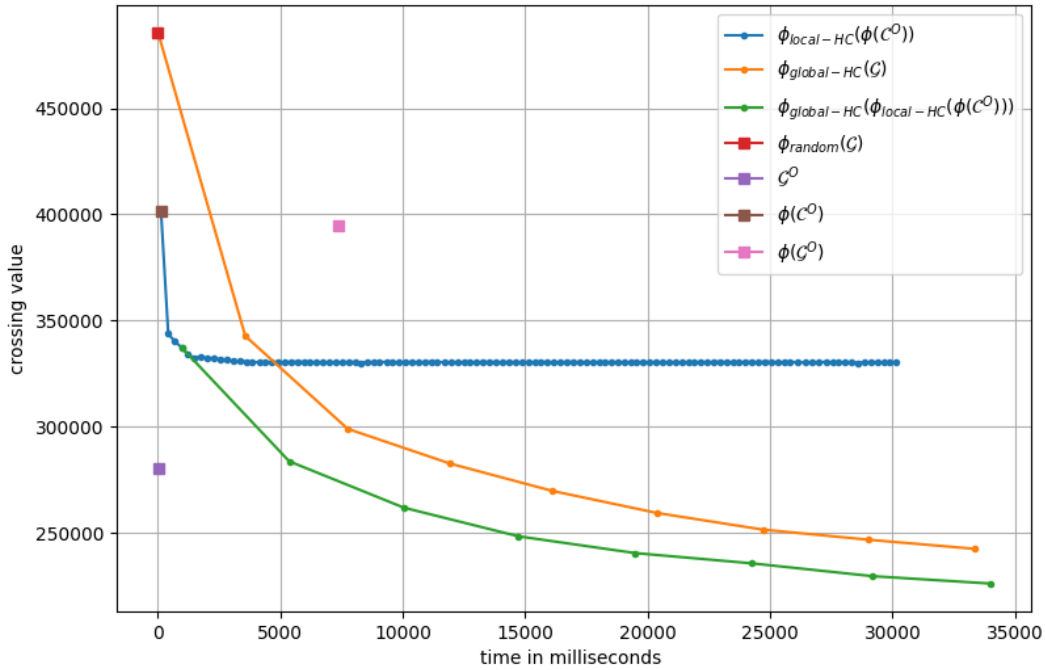


Figure 3.1: Illustration of the global hill-climbing and our proposed algorithm in regard to the time.

(green line). In the Experiment 3.1 we also illustrate the mapping performance without clustering (pink) and the performance of the spring embedding (purple).

3.1.2 Graph Clustering

Recursive vs. Iterative Clustering

The success of the graph clustering can be measured with the crossings of the weighted cluster graph $\mathcal{G}_W = (V_G, E_G)$, where V_G represents the number of graph clusters. To compute the success of a clustering we calculate the total number of crossings of the optimized weighted cluster Graph $\mathcal{G}_W^O$ in regard to the amount of clusters.

$$\frac{1}{V_G} \cdot cv(\mathcal{G}_W)$$

To determine whether to use the iterative graph clustering (see 2) or the recursive graph clustering (see 1), we conduct two Experiments illustrated in Figure 3.2 and 3.3:

1. **Cluster Success in regard to Edge Probability**
   In the Experiment 3.2 the success of the clustering was tested on multiple

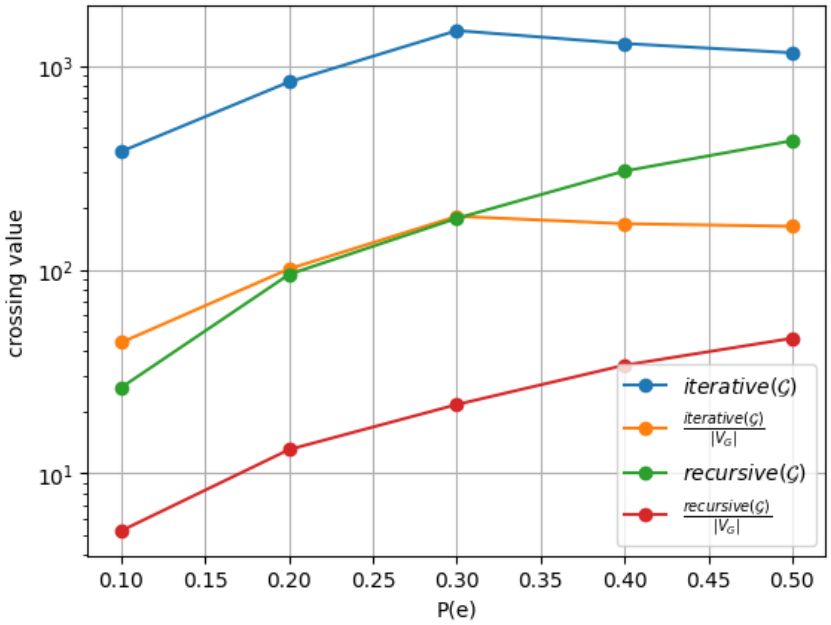


Figure 3.2: Test the clustering success of recursive and iterative clustering with respect to the edge probability.

random generated graphs with different sizes and the edge probabilities shown on the x-axis. The edge probability $P(e)$ is the probability of two arbitrary nodes to be connected with an edge.

$$P(e) = P((u,v)) = P(u,v \in V, \exists\ e \in E \mid e = (u,v))$$

2. **Cluster Success in regard to the Graph Size**
   In the Experiment 3.3 the success of the clustering was tested in regard to multiple random graphs of different size (number of vertices).

3.1.3 Mapping

The mapping procedure incorporates the optimization with the spring embedding and the minimum distance matching. To gain deeper insights into the mapping process, we analyzed its performance under conditions with and without clustering, therefore also gaining information about the performance of the clustering. For the evaluation, we used the following setups:

1. **Relationship Between $|\mathcal{P}|$ and $|V|$**
   Experiment 3.4 examines the correlation between the size of the point set $|\mathcal{P}|$ and the number of nodes in the graph $|V|$, providing insights

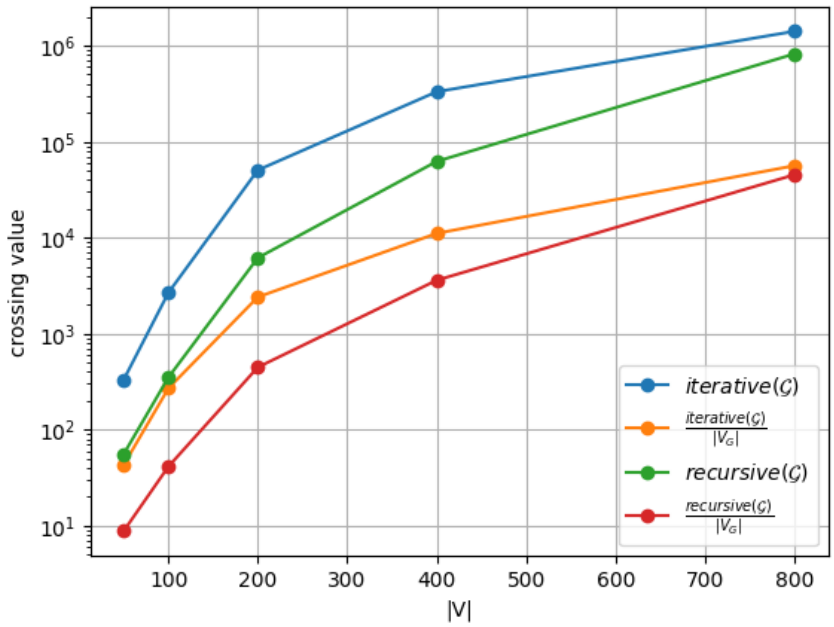


Figure 3.3: Test the clustering success of recursive and iterative clustering with different graph sizes.

into how the cardinality of the point set affects the mapping quality and computational complexity.

2. **Variation in the Number of Nodes $|V|$**
   Experiment 3.5 evaluates the performance of the mapping process as the number of nodes $|V|$ increases. By analyzing different graph sizes, we assess scalability and identify potential limitations in the mapping.

For each of the above experiments the crossing value will be assessed in four distinct states.

I: The crossing value after a random assignment (blue).

$$cv(\phi_{\texttt{random}}(G))$$

II: The crossing value after the optimization with the spring embedding (orange).

$$cv(\mathcal{G}^O)$$

III: The crossing value after mapping $\mathcal{G}^O$ onto $\mathcal{P}$ with the minimum distance matching (green).

$$cv(\phi(\mathcal{G}^O))$$

IV: The crossing value after mapping all graph clusters $\mathcal{C}^O$ onto their regarding point cluster $\mathcal{P}_C$ with the minimum distance matching (red).

$$cv(\phi(\mathcal{C}^O))$$

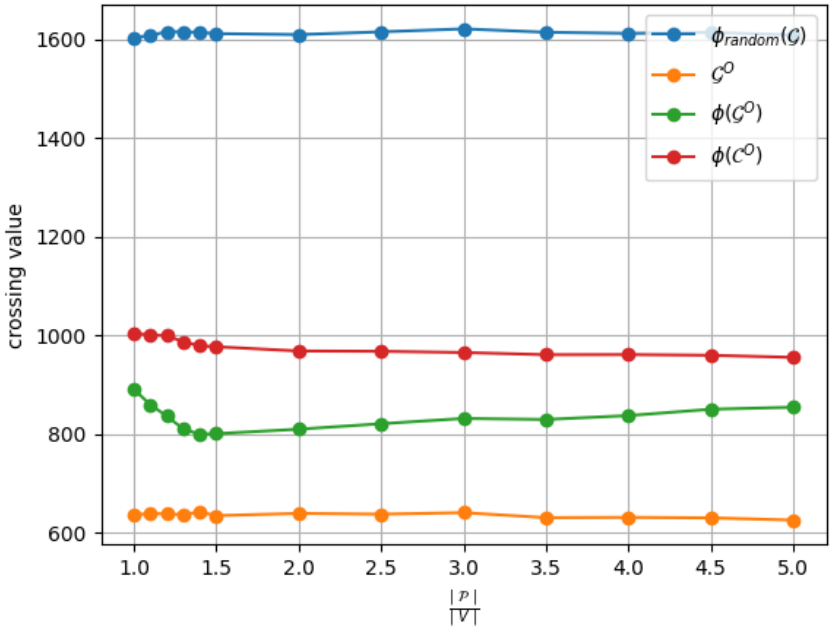


Figure 3.4: This figure illustrates the crossing minimization performance in the mapping process with respect to the relationship of $|\mathcal{P}|$ to $|V|$.

3.1.4 Algorithm

Finally we want to measure the performance of our proposed algorithm on different types of graphs and point sets. To further investigate our algorithm the crossing value $cv(\cdot)$ will be assessed in four distinct states.

I: The crossing value after a random assignment.

$$cv(\phi_{\texttt{random}}(G))$$

II: The crossing value after mapping all clusters.

$$cv(\text{map}(\phi(\mathcal{C}^O)))$$

III: The crossing value of $\mathcal{G}$ after application of the local hill-climbing

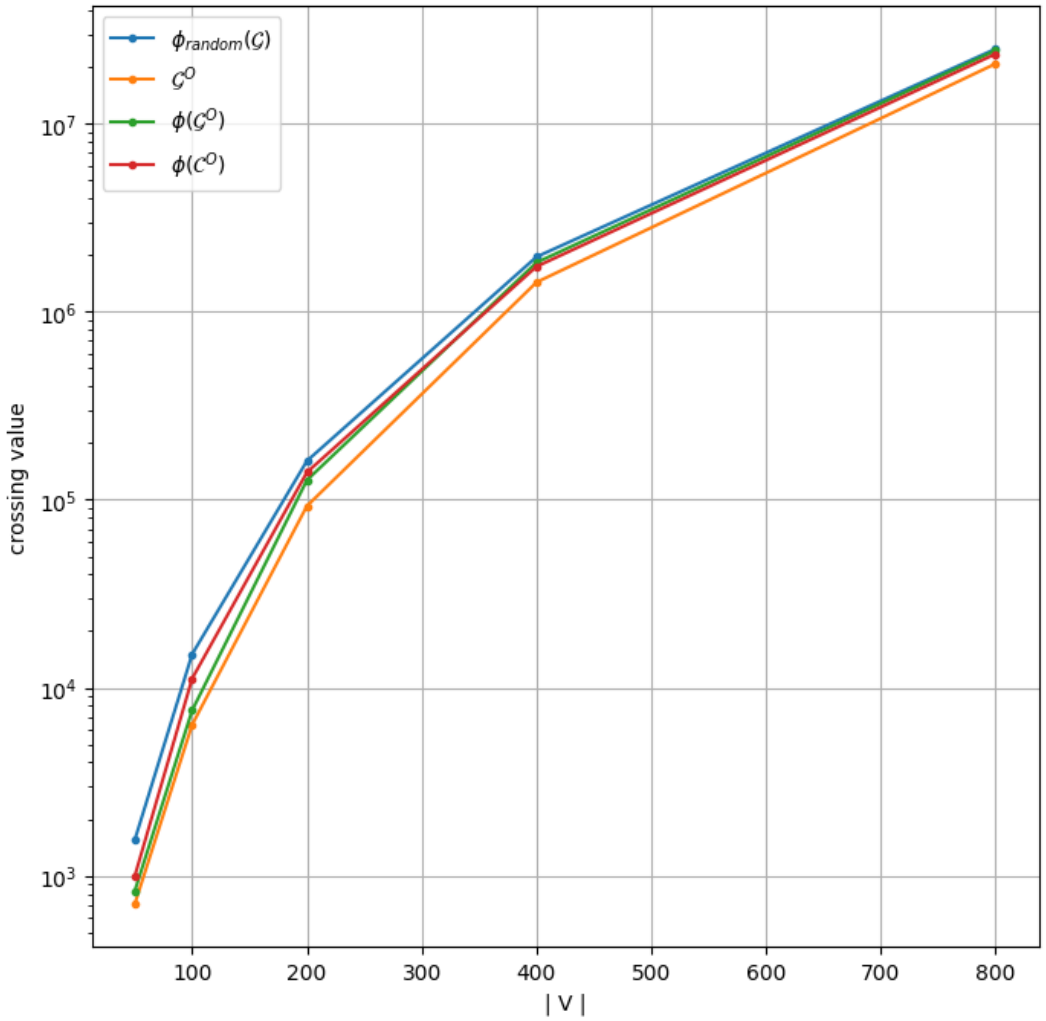


Figure 3.5: This figure illustrates the crossing minimization performance in the mapping process with respect to $|V|$.

method to all clusters.

$$cv(\text{map}(\phi_{\texttt{local-HC}}(\mathcal{C})))$$

IV: The crossing value after application of the global hill-climbing method.

$$cv(\phi_{\texttt{global-HC}}(\mathcal{G}))$$

Figure 3.6 illustrates the behavior of the crossing values across these four stages, analyzed with respect to the ratio $\frac{|\mathcal{P}|}{|V|}$. To further examine the hill-climbing methods, the experiment also investigates the effect of varying $P(e)$, as shown in Figure 3.7.

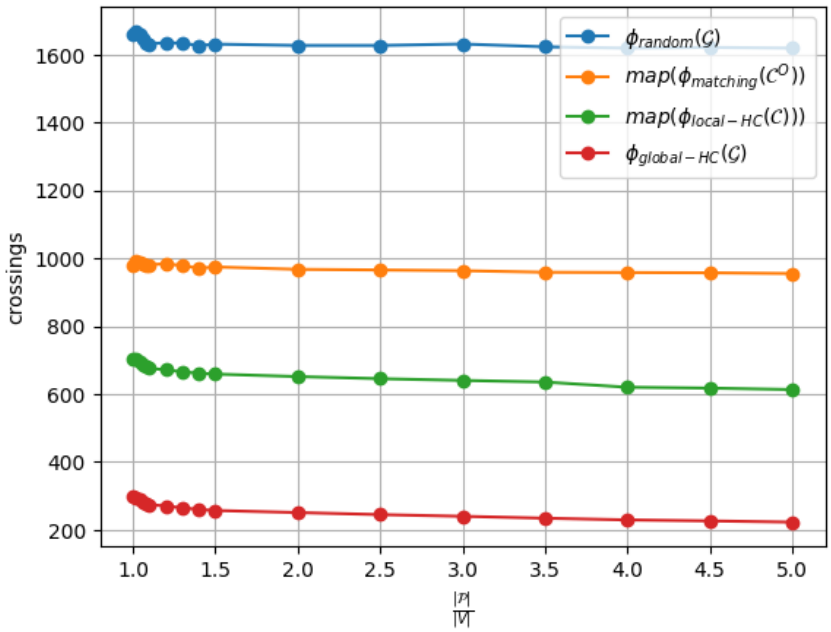


Figure 3.6: Measurement of success of crossing minimization in regard to $\frac{|\mathcal{P}|}{|V|}$ in four distinct stages 3.1.4: I ← (blue), II ← (orange), III ← (green) and IV ← (red).

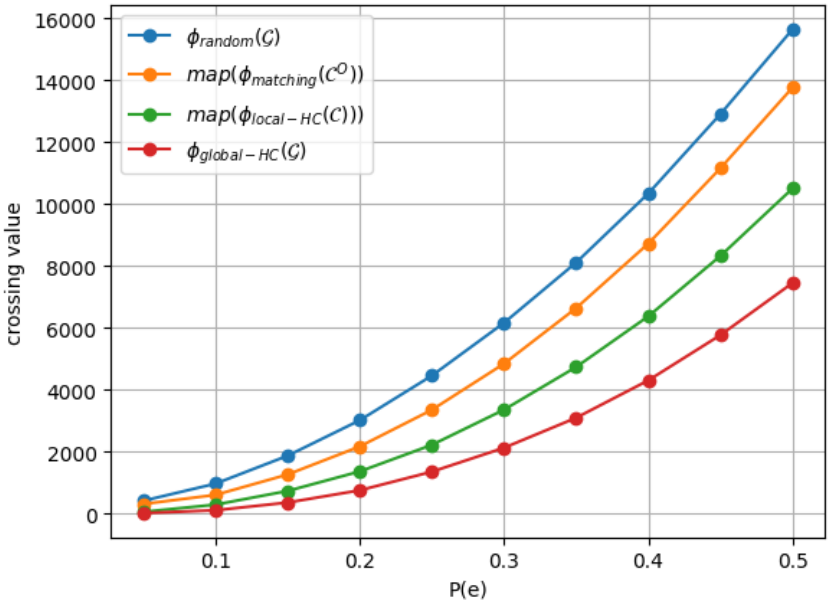


Figure 3.7: Measurement of crossing minimization performance in regard to P(e) in four distinct stages 3.1.4: I ← (blue), II ← (orange), local hill-climbing method without switch operation ← (green) and IV ← (red).

3.2 Analysis

This chapter presents a comprehensive analysis of the data collected to address the research questions and objectives outlined in earlier chapters.

3.2.1 Time Complexity

In Figure 3.1, the performance of a graph comprising 200 nodes is presented with respect to computation time. The results highlight the advantages of the proposed algorithm compared to the standalone global hill-climbing method. The incorporation of clustered mapping and the local hill-climbing approach results in a substantial reduction in computation time. Moreover, as the graphs grow in size and density—thereby increasing computational complexity—the performance gap between the proposed algorithm and the global hill-climbing method widens.

3.2.2 Graph

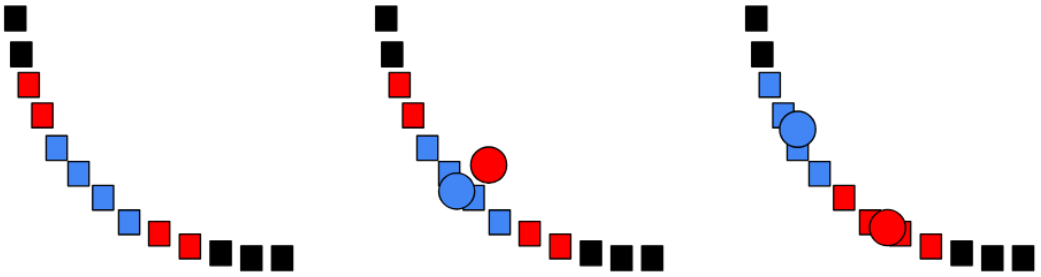
Recursive vs. Iterative Graph Clustering

The results presented in Section 3.1.2 indicate that the recursive approach to graph cluster consistently outperforms its iterative counterpart, particularly for input graphs $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ characterized by relatively small vertex ($|V|$) and edge ($|E|$) counts. However, the recursive approach is significantly slower in terms of computation time. The choice of an appropriate threshold for selecting between the two methods is highly dependent on the available computational resources. In our experiments, a threshold of 100,000 edges was used to balance performance and efficiency. For very large graphs, the performance of the recursive and iterative approaches converges, exhibiting comparable results.

3.2.3 Point Set

Certain Point Sets

1. **Circle**
   On circular patterns the local point set clustering can lead to some problems, especially with large point sets and numerous clusters. An optimal solution (see 3.8c) for a point clustering on a circle would be to place all the clusters next to each other on the circle, but our algorithm will place the point clusters as shown in Figure 3.8a. Both clusters (blue and red) resemble convex hulls but the centroids of these clusters are quite misleading (see 3.8b). Besides that, every edge of the blue cluster to another



(a) Clustering result for the bottom left of a circular point set, highlighting the red and blue clusters.

(b) Centroids (blue and red circles) resulting from the clustering in Figure 3.8a.

(c) A better clustering solution for subsequent matching processes.

Figure 3.8: Illustration of challenges with the local point clustering algorithm on large circular point sets, focusing on the bottom left corner of the circle.

cluster will pass the convex hull of the red cluster. This demonstrates that large circles can lead to unwanted behavior.

2. **Grid**
   Grid-like point sets introduce distinct challenges for the local point clustering algorithm due to their regular collinear structure. The algorithm forms clusters without paying special attention to collinear points. Consequently this leads to many collinear points. Especially the sides of the grid are endangered to get clustered into one straight line, which later on leads to many collinear crossing penalties. This demonstrates how heavily we rely on the hill-climbing method, which effectively reduces collinear crossings.

3. **Line**
   A line can be seen as a special type of grid. The same issues encountered with grid-like structures also apply here. Since the local point clustering algorithm does not explicitly handle collinear points, its performance on linear point sets is limited. This leads to suboptimal clustering results and increased crossing penalties in the subsequent matching steps.

4. **Random Distribution**
   The matching algorithm generally performs well on random point sets, particularly when there are few collinear points. When the point distribution includes a significant number of collinear points, resulting in many collinear crossing penalties, the hill-climbing optimization method effectively reduces these crossings. This demonstrates the robustness of the algorithm in such scenarios.

3.2.4 Local Point Clustering

Collinear Points

It would be an option to add certain constraints to the local point cluster algorithm in order to reduce the number of collinear points within a point cluster, but after the matching we still apply the hill-climbing method, where collinear crossings lead to very large penalties. Consequently, collinear crossings are then removed with high priority. Therefore, it is not really necessary to add a collinear constraint to the local cluster algorithm. The constraint would also negatively affect the success of the local point clustering algorithm in regard to creating convex hulls.

Advantages
The algorithm is relatively fast with hard constraints like the fixed number of clusters and the fixed cluster sizes. Besides that it has shown to perform well on random distributed point sets.

Disadvantages
The simplicity of the point clustering approach can result in suboptimal performance for certain geometric configurations. For instance, the algorithm tends to perform poorly on point sets arranged in a circular pattern or grid-like point structures.

3.2.5 Mapping

Number of Points $|\mathcal{P}|$

The results presented in Figure 3.4 illustrate the performance differences in the matching process with respect to the ratio $\frac{|\mathcal{P}|}{|V|}$. The Experiment 3.4 was conducted on relatively small graphs. Notably, there is a pronounced improvement in performance, specifically a reduction in crossings in $\phi(\mathcal{G}^O)$, with the addition of only a few extra points to $\mathcal{P}$. This finding highlights that even a small increase in the size of the point set can lead to a significant enhancement in the representation of the layout derived from the spring embedding. On the other hand the clustering-based approach $\phi(\mathcal{C}^O)$ demonstrates worse performance than $\phi(\mathcal{G}^O)$. But remains more stable in regard to the ratio $\frac{|\mathcal{P}|}{|V|}$. It is important to note that the experimental setup maintained a consistent plane size across all point set sizes. Consequently, for larger point sets, collinearity among points became increasingly probable. In an idealized point set devoid of collinear points, the crossing value $cv(\mathcal{G}^O)$ would asymptotically approach $cv(\mathcal{G}^O)$ as the size of $\mathcal{P}$ increases. This behavior reflects the improved ability of the mapping to more accurately represent the layout of the spring embedding with each additional point in the set.

3.2.6 Clustering-based Mapping $\phi(\mathcal{C}^O)$

As illustrated in Figure 3.5, an increase in the number of vertices in $\mathcal{G}$ is strongly associated with a decline in mapping performance. Large graphs pose significant challenges to the mapping process. The absence of clustering in these experiments exacerbates this issue. Specifically, the mapping $\phi(\mathcal{G}^O)$ without clustering demonstrates superior performance compared to the clustering-based mapping $\phi(\mathcal{C}^O)$ for smaller graphs. However, for larger graphs, the performance of $\phi(\mathcal{G}^O)$ degrades and is even surprisingly surpassed by $\phi(\mathcal{C}^O)$. Additionally, as demonstrated in Figure 3.1, $\phi(\mathcal{C}^O)$ is significantly faster than $\phi(\mathcal{G}^O)$, particularly for larger graphs. For smaller graphs, the clustering process represents a trade-off, offering reduced computational complexity at the cost of diminished crossing minimization performance. These findings highlight the pivotal role of clustering in enhancing the efficiency and scalability of the mapping process, particularly for large graphs.

3.2.7 Algorithm

In Figures 3.6 and 3.7 the performance of our algorithm is evaluated. In 3.6 the performance of different stages are showcased. It is noticeable that in the hill-climbing the performance increases slightly with the $\frac{|\mathcal{P}|}{|V|}$ ratio. This is due to the fact that the number of possible drawings increase. The Experiment 3.7 underlines the robustness of our algorithm in regard to dense graphs.

3.3 Conclusion

In this Chapter, we have presented a comprehensive evaluation of the proposed algorithm for minimal point set crossing drawings through detailed experiments and analyses. The results highlight the algorithm's capability to minimize edge crossings effectively across various graph structures and configurations. Key insights include the benefits of recursive clustering for smaller graphs, the challenges posed by specific point set geometries, and the importance of incorporating clustering and local hill-climbing optimization to handle large graphs efficiently. Furthermore, the robustness of the hill-climbing optimization method demonstrates its critical role in achieving high-quality drawings.

Chapter 4

Discussion and Outlook

4.1 Research Context and Objectives

This thesis aimed to minimize edge crossings in graph drawings over fixed point sets. The main objective was to find an initial mapping, for the *minimal point set crossing embedding problem*, from where a rather common hill-climbing approach can be started. This is especially interesting for very large graphs which are computationally intense.

4.2 Interpretation of Key Findings

- **Mapping:**
  A significant reduction in the crossing number is observed when combining the spring embedding approach (see 2.3) with our minimum distance matching technique (see 2.4). The mapping performs particularly well when the condition $|\mathcal{P}| > |\mathbf{V}|$ holds.
  Moreover, the mapping process is computationally much faster than traditional hill-climbing methods, especially when employing the greedy minimum distance matching algorithm (see 3).

- **Local Point Clustering:**
  Another novel approach is our local point clustering algorithm (see 2.2), which focuses on clustering points within convex hulls. The algorithm demonstrates efficient clustering performance under specific size constraints imposed by the graph clustering.

- **Local Hill Climbing:**
  To reduce the computational overhead we introduce the local hill-climbing method (see 2.5.2). This enables us to faster reduce a large

amount of edge crossings before the computational more expensive global hill-climbing method (see 2.5.3) is applied.

In conclusion, our algorithm incorporates efficient preprocessing steps that enhance the performance of the global hill-climbing algorithm by significantly reducing computational overhead.

4.3 Critical Evaluation of the Work

The proposed algorithm successfully reduces crossings. However, certain limitations exist:

- **Certain Point Sets:**
  The algorithm faces challenges with different types of point sets. For circular patterns, clustering leads to misleading centroids. Grid-like structures cause collinear points to be clustered particularly along the grid sides. While the algorithm tends to perform well on random distributions, especially the mapping struggles when many collinear points are present, though the hill-climbing method helps reduce these penalties.

- **Unstable Maximum Weighted Matching:**
  Unfortunately, our algorithm implements the rather unstable boost maximum weighted matching algorithm and a greedy minimum distance matching as a fallback solution. More precise experiments could have been conducted with a more stable maximum weighted matching.

- **Local Optima in Hill-Climbing:**
  Our experiments did not provide evidence that the mapping technique employed always leads to a faster convergence of the hill-climbing method. In fact, the mapping could introduce local extrema in the hill-climbing process. This occurs when no further move or switch operations can be made without increasing the crossing number, even though the global optimum has not yet been reached.

- **Centroid Approximation:**
  The graph clustering produces a random number of clusters, each with a random size. Subsequently, the positions (centroids) of the weighted cluster graph ($\mathcal{G}_W$) are approximated using the local point clustering algorithm, while assuming equal cluster sizes. This approach does not account for the individual cluster sizes determined by the graph clustering, potentially leading to poor centroid approximations and, consequently, degraded performance in later stages.

4.4 Comparison with Related Work

Our proposed approach diverges from traditional heuristics by focusing on a computationally efficient preprocessing phase (see section 3.1.3). Specifically, we employ a combination of spring embedding [21] (see fig. 2.3) and minimum distance matching (see section 2.4) to generate an initial mapping that significantly reduces the crossing number before iterative optimization methods like hill-climbing [27](see sections 2.5.2 and 2.5.3) are applied. Unlike methods such as genetic algorithms [25, 26] or simulated annealing [22, 29], which directly aim for global optima, our technique emphasizes rapid initial improvement, particularly for large graphs.
For example, in comparison to star insertion [9], our method does not rely on vertex-by-vertex insertion but instead leverages global optimization techniques [21] in the initial mapping phase. Similarly, while simulated annealing [22, 29] and genetic algorithms [25, 26] are more exhaustive in exploring the solution space, our clustering algorithms introduces a novel way by reducing the problem to smaller graphs with smaller point sets.
This highlights that, while our method shares similarities with heuristic approaches like hill-climbing [27], it introduces preprocessing strategies that complement and enhance existing frameworks. These innovations contribute to reducing computational overhead, particularly in large-scale scenarios, while maintaining competitive performance in minimizing crossings. However, as discussed earlier, certain limitations, such as challenges with collinear point sets and local optima, suggest potential areas for refinement in future work.

4.5 Future Work

Based on the findings and limitations, several directions for future research are recommended:

- Explore heuristics that directly tackle collinear crossings and specific point sets.

- Investigate factors that lead to local extremes when applying hill-climbing algorithm and explore the usability of simulated annealing [22], instead of the proposed hill-climbing method, to avoid local optima.

- Extend the approach to use a same sized graph clustering or a more fine grained estimation of the centroids.

4.6 Concluding Remarks

In conclusion, this chapter has discussed the significance of the proposed algorithm in minimizing edge crossings in straight-line graph drawings over fixed point sets. The findings highlight the effectiveness of combining spring embedding with a minimum distance matching technique to provide a fast initial mapping of $\mathbf{V}$ to $\mathcal{P}$. Besides that we introduced a local hill-climbing method which needs less computation than traditional methods. Furthermore, the introduction of local point clustering and its application has shown promising results in clustering performance, particularly in specific size-constrained scenarios. Despite these strengths, challenges remain, especially regarding the handling of specific point sets and the evaluation of the algorithm regarding local extremes in hill-climbing.
The work presented here lays the groundwork for future improvements and provides a novel approach to the minimal point set crossing embedding problem, which can be particularly valuable in large-scale graph visualization and other real-world applications such as network design, geographic mapping, and VLSI layout optimization. Further research is needed to refine the clustering process and address the limitations identified, ultimately leading to more robust and efficient solutions for the minimal point set crossing embedding problem.

Bibliography

[1] David Auber, Yves Chiricota, Fabien Jourdan, and Guy Melançon. Multiscale visualization of small world networks. volume 3, 01 2003.

[2] Andre Barreto and Helio Barbosa. Graph layout using a genetic algorithm. pages 179–184, 01 2000.

[3] Chris Bennett, Jody Ryall, Leo Spalteholz, and Amy Gooch. The aesthetics of graph visualization. In *Proceedings of the Third Eurographics Conference on Computational Aesthetics in Graphics, Visualization and Imaging*, Computational Aesthetics'07, page 57–64, Goslar, DEU, 2007. Eurographics Association.

[4] Daniel Bienstock and Nathaniel Dean. Bounds for rectilinear crossing numbers. *Journal of Graph Theory*, 17(3):333–348, 1993.

[5] Aaron Büngener and Michael Kaufmann. Improving the crossing lemma by characterizing dense 2-planar and 3-planar graphs. In *32nd International Symposium on Graph Drawing and Network Visualization (GD 2024)*, pages 29–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.

[6] H.-F.S. Chen and D. Lee. On crossing minimization problem. *Computer-Aided Design of Integrated Circuits and Systems, IEEE Transactions on*, 17:406–418, 05 1998.

[7] Robin Christian, R Bruce Richter, and Gelasio Salazar. Zarankiewiczs conjecture is finite for each fixed m. *Journal of Combinatorial Theory, Series B*, 103(2):237–247, 2013.

[8] Robert J Cimikowski. An analysis of some heuristics for the maximum planar subgraph problem. In *SODA*, volume 95, pages 322–331. Citeseer, 1995.

[9] Kieran Clancy, Michael Haythorpe, and Alex Newcombe. An effective crossing minimisation heuristic based on star insertion, 2019.

[10] Hubert De Fraysseix, János Pach, and Richard Pollack. How to draw a planar graph on a grid. *Combinatorica*, 10:41–51, 1990.

[11] Alexander Dobler, Martin Nöllenburg, Daniel Stojanovic, Anaïs Villedieu, and Jules Wulms. Crossing minimization in time interval storylines, 2023.

[12] Stephane Durocher and Debajyoti Mondal. On the hardness of point-set embeddability. In *International Workshop on Algorithms and Computation*, pages 148–159. Springer, 2012.

[13] Peter Eades. A heuristic for graph drawing, 1984.

[14] Jack Edmonds. Maximum matching and a polyhedron with 0,1-vertices. *Journal of Research of the National Bureau of Standards Section B Mathematics and Mathematical Physics*, page 125, 1965.

[15] Timo Eloranta and Erkki Mäkinen. Timga: A genetic algorithm for drawing undirected graphs. *Divulgaciones Matemáticas*, 9(2):155–170, 2001.

[16] Qing-Wen Feng, Robert F. Cohen, and Peter Eades. Planarity for clustered graphs. In Paul Spirakis, editor, *Algorithms — ESA '95*, pages 213–226, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.

[17] Harold N. Gabow. A data structure for nearest common ancestors with linking, 2016.

[18] M. R. Garey and D. S. Johnson. Crossing number is np-complete. *SIAM Journal on Algebraic Discrete Methods*, 4(3):312–316, 1983.

[19] Loann Giovannangeli, Frédéric Lalanne, David Auber, Romain Giot, and Romain Bourqui. Deep neural network for drawing networks. In *GD*, volume 12868 of *Lecture Notes in Computer Science*, pages 375–390. Springer, 2021.

[20] Fáry István. On straight-line representation of planar graphs. *Acta scientiarum mathematicarum*, 11(229-233):2, 1948.

[21] Tomihisa Kamada and Satoru Kawai. An algorithm for drawing general undirected graphs. *Information Processing Letters*, 31(1):7–15, 1989.

[22] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983.

[23] Frank Thomson Leighton. New lower bound techniques for vlsi. *Mathematical systems theory*, 17:47–70, 1984.

[24] Richard J Lipton and Robert Endre Tarjan. A separator theorem for planar graphs. *SIAM Journal on Applied Mathematics*, 36(2):177–189, 1979.

[25] Alexey S Rodionov, Hyunseung Choo, and Kseniya A Nechunaeva. Framework for biologically inspired graph optimization. In *Proceedings of the 5th International Conference on Ubiquitous Information Management and Communication*, pages 1–4, 2011.

[26] Alexey S Rodionov and Kseniya A Nechunaeva. Network structure optimization: genetic operators: mutation and crossover. In *Proceedings of the 7th International Conference on Ubiquitous Information Management and Communication*, pages 1–3, 2013.

[27] Alejandro Rosete-Suárez, Alberto Ochoa-Rodríguez, and Michele Sebag. Automatic graph drawing and stochastic hill-climbing. In *Genetic and Evolutionary Computation Conference*, pages 1699–1706. Morgan Kaufmann, 1999.

[28] Marcus Schaefer. The graph crossing number and its variants: A survey. *The electronic journal of combinatorics*, pages DS21–May, 2012.

[29] Peter JM Van Laarhoven, Emile HL Aarts, Peter JM van Laarhoven, and Emile HL Aarts. *Simulated annealing*. Springer, 1987.

Selbständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Bachelorarbeit selbständig und nur mit den angegebenen Hilfsmitteln angefertigt habe und dass alle Stellen, die dem Wortlaut oder dem Sinne nach anderen Werken entnommen sind, durch Angaben von Quellen als Entlehnung kenntlich gemacht worden sind. Diese Bachelorarbeit wurde in gleicher oder ähnlicher Form in keinem anderen Studiengang als Prüfungsleistung vorgelegt.

Tübingen, 18.12.2024
Ort, Datum

Bela Thraen
Unterschrift